AI ASSISTED CODING

NAME: V.Nageshwari

BATCH: 39

ROLL NO: 2303A52063

Lab 9 – Documentation Generation: Automatic Documentation and Code Comments

Task Description -1 (Documentation – Function Summary Generation)

Task:

Use AI to generate concise functional summaries for each Python function in a given script.

Instructions:

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details.

Expected Output -1:

A Python script where each function contains a clear and concise summary explaining its purpose.

PROMPT:

#generate a python script.
#and then write a short summary describing the purpose of each function.
#Exaplain onlythe purpose of the functions without going into the logic of how they work.and also keep the summaries technically accurate and then add the summary as a docstring inside each function.

CODE:

```
lab 9.1_task 1.ipy > ...
#generate a python script.
#and then write a short summary describing the purpose of each function.
#Exaplain onlythe purpose of the  functions without going into the logic of how they work.and also keep the summaries technically accurate and then add the summary as a docstri
def is_even(num):
    """
    This function checks if a given number is even. It takes an integer as input and returns True if the number is even, otherwise it returns False.
    """
    if num % 2 == 0:
        return True
    else:
        return False
def count_even_numbers(numbers):
    """
    This function counts the number of even numbers in a given list of integers. It takes a list of integers as input and returns the count of even numbers found in that list.
    """
    count = 0
    for num in numbers:
        if is_even(num):
            count += 1
    return count
#Example usage:
numbers = [1, 2, 3, 4, 5, 6]
print("Count of even numbers:", count_even_numbers(numbers)) # Output: 3
numbers = [10, 15, 20, 25, 30]
print("Count of even numbers:", count_even_numbers(numbers)) # Output: 3
numbers = [0, -2, -4, -6, -8]
print("Count of even numbers:", count_even_numbers(numbers)) # Output: 5
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\Python AI> & C:/Users/DELL/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DELL/Desktop/Python AI/lab 9.1_task 1.ipy"
Count of even numbers: 3
Count of even numbers: 3
Count of even numbers: 5
```

ANALYSIS:

- The prompt instructs AI to perform function-level documentation, focusing only on the purpose of each Python function.
- AI interprets this as a code summarization task, not code explanation, modification, or optimization.
- Constraints like concise, technically accurate, and no implementation details guide AI to generate brief, high-level summaries only.
- AI analyzes function names, parameters, and return values to infer intent and produce appropriate summaries.
- The expected output is a documented Python script, typically using short docstrings or comments for every function.

Task Description -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program.

Instructions:

• Provide a Python program without comments.

• Instruct AI to explain only decision-making logic and loop behavior.

• Skip basic syntax explanations.

Expected Output -2:

Python code with clear explanations describing the logic of conditions and loops.

PROMPT:

#Write a python program to document the logic behind conditional statements and loops.
This function checks if a given number is positive. It takes an integer as input and returns True if the number is greater than zero, otherwise it returns False.
This function counts the number of positive numbers in a given list of integers. It takes a list of integers as input and returns the count of positive numbers found in that list.
#explain only decision-making logic and loop behavior.
#Skip basic syntax explanations.

CODE:

lab 9.1_task 2.ipy > ...

```python
#Write a python program to document the logic behind conditional statements and loops.
def is_positive(num):
    # This function checks if a given number is positive. It takes an integer as input and returns True if the number is greater than zero, otherwise it retu
    if num > 0:
        return True
    else:
        return False
def count_positive_numbers(numbers):
    # This function counts the number of positive numbers in a given list of integers. It takes a list of integers as input and returns the count of positive
    count = 0
    for num in numbers:
        if is_positive(num):
            count += 1
    return count
# Example usage:
numbers = [-1, 0, 1, 2, 3, -2]
print("Count of positive numbers:", count_positive_numbers(numbers)) # Output: 3
numbers = [10, -5, 20, -15, 30]
print("Count of positive numbers:", count_positive_numbers(numbers)) # Output: 3
numbers = [-10, -20, -30, -40, -50]
print("Count of positive numbers:", count_positive_numbers(numbers)) # Output: 0
numbers = [0, 0, 0, 0, 0]
print("Count of positive numbers:", count_positive_numbers(numbers)) # Output: 0
numbers = [5, 10, 15, 20, 25]
print("Count of positive numbers:", count_positive_numbers(numbers)) # Output: 5
#explain only decision-making logic and loop behavior.
#Skip basic syntax explanations.
# In the `is_positive` function, we use a conditional statement (if-else) to determine whether the input number is greater than zero. If the condition `num >
# In the `count_positive_numbers` function, we initialize a counter variable `count` to zero. We then use a for loop to iterate through each number in the in
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\Python AI> & C:/Users/DELL/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DELL/Desktop/Python AI/lab 9.1_task 2.
Count of positive numbers: 3
Count of positive numbers: 3
Count of positive numbers: 0
Count of positive numbers: 0
Count of positive numbers: 5
```

ANALYSIS:

- The prompt asks AI to perform logic-level documentation, focusing specifically on conditional statements (if, elif, else) and loop structures (for, while).
- AI understands that it must explain the decision-making logic and iteration behavior, not rewrite or modify the code.
- The instruction to skip basic syntax explanations tells AI to avoid describing what if or for means and instead explain why the condition exists and what the loop achieves logically.
- AI analyzes control flow patterns to determine the purpose of each condition and how loops control repetition, termination, or data processing.
- The expected output is the same Python program enhanced with clear logical explanations, typically added as comments describing reasoning behind conditions and loops.

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of an entire Python file.

Instructions:

• Provide the complete Python file to AI.
• Ask AI to write a brief overview summarizing the file's purpose and functionality.
• Place the overview at the top of the file.

Expected Output -3:

A Python file with a clear and concise file-level overview at the Beginning.

PROMPT:

#write a python program to generate a high-level overview describing the functionality of an entire Python file.

CODE:

```
lab 9.1_task 3.ipy > ...
#write a python program to generate a high-level overview describing the functionality of an entire Python file.
"""
This Python file contains two functions: `is_even` and `count_even_numbers`. The `is_even` function checks if a given number is even, while the `count_even_n
"""
def is_even(num):
    """
    This function checks if a given number is even. It takes an integer as input and returns True if the number is even, otherwise it returns False.
    """
    if num % 2 == 0:
        return True
    else:
        return False
def count_even_numbers(numbers):
    """
    This function counts the number of even numbers in a given list of integers. It takes a list of integers as input and returns the count of even numbers f
    """
    count = 0
    for num in numbers:
        if is_even(num):
            count += 1
    return count

    #Example usage:
numbers = [1, 2, 3, 4, 5, 6]
print("Count of even numbers:", count_even_numbers(numbers)) # Output: 3
numbers = [86,45, 20, 25, 30]
print("Count of even numbers:", count_even_numbers(numbers)) # Output: 3
numbers = [0, -2, -4, -6, -8]
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\Python AI> & C:/Users/DELL/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DELL/Desktop/Python AI/lab 9.1_task 3.ip
Count of even numbers: 3
Count of even numbers: 3
```

ANALYSIS:

- The prompt asks AI to generate a file-level overview, meaning a high-level summary of the entire Python script rather than individual functions or logic blocks.
- AI understands this as a global summarization task, where it must analyze the overall structure, main components, and primary purpose of the file.
- The instruction to keep it brief and concise guides AI to produce a short paragraph that summarizes functionality without detailing internal logic or implementation steps.
- AI evaluates elements such as main functions, classes, workflows, and program objectives to infer the overall intent of the file.

- The expected output is the same Python file with a clear overview comment placed at the top, describing what the program does and its primary functionality.

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

Instructions:

• Provide Python code containing basic or unclear comments.
• Ask AI to rewrite the documentation to improve clarity and consistency.
• Ensure technical meaning remains unchanged.

Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

PROMPT:
#Documentation – Refine Existing Documentation.
#Provide Python code containing basic or unclear comments.
#and then rewrite the documentation to improve clarity and consistency.and also .
#Ensure technical meaning remains unchanged.
#Example of basic or unclear comments #Improved documentation:

CODE:

lab 9.1_task 4.ipy > ...

```python
#Documentation – Refine Existing Documentation.
#Provide Python code containing basic or unclear comments.
#and then rewrite the documentation to improve clarity and consistency.and also •   Ensure technical meaning remains unchanged.
#Example of basic or unclear comments:
def calculate_area(radius):
    # This function calculates the area of a circle given its radius.
    area = 3.14 * radius * radius
    return area
#Improved documentation:
def calculate_area(radius):
    """
    This function calculates the area of a circle based on the provided radius. It uses the formula A = πr², where A is the area and r is the radius. The func
    """
    area = 3.14 * radius * radius
    return area
# Example usage:
radius = 5
print("Area of the circle:", calculate_area(radius)) # Output: Area of the circle: 78.5
#test the function with different radius values to ensure it works correctly.
radius = 10
print("Area of the circle:", calculate_area(radius)) # Output: Area of the circle:
314.0
radius = 0
print("Area of the circle:", calculate_area(radius)) # Output: Area of the circle:
0.0
```

OUTPUT:

```
PS C:\Users\DELL\Desktop\Python AI> & C:/Users/DELL/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DELL/Desktop/Python AI/lab 9.1_task 4.ipy
 Area of the circle: 78.5
 Area of the circle: 314.0
 Area of the circle: 0.0
```

ANALYSIS:

- The prompt asks AI to perform documentation refinement, meaning it must improve existing comments without changing the actual code or technical meaning.
- AI understands this as an editing and clarity enhancement task, not code modification or logic correction.
- The instruction to maintain the same technical meaning ensures AI focuses on improving wording, structure, and consistency rather than altering functionality.
- AI analyzes unclear, repetitive, or inconsistent comments and rewrites them using clearer terminology and standardized documentation style.
- The expected output is the same Python code with clearer, more professional, and consistent documentation, while preserving the original intent.

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

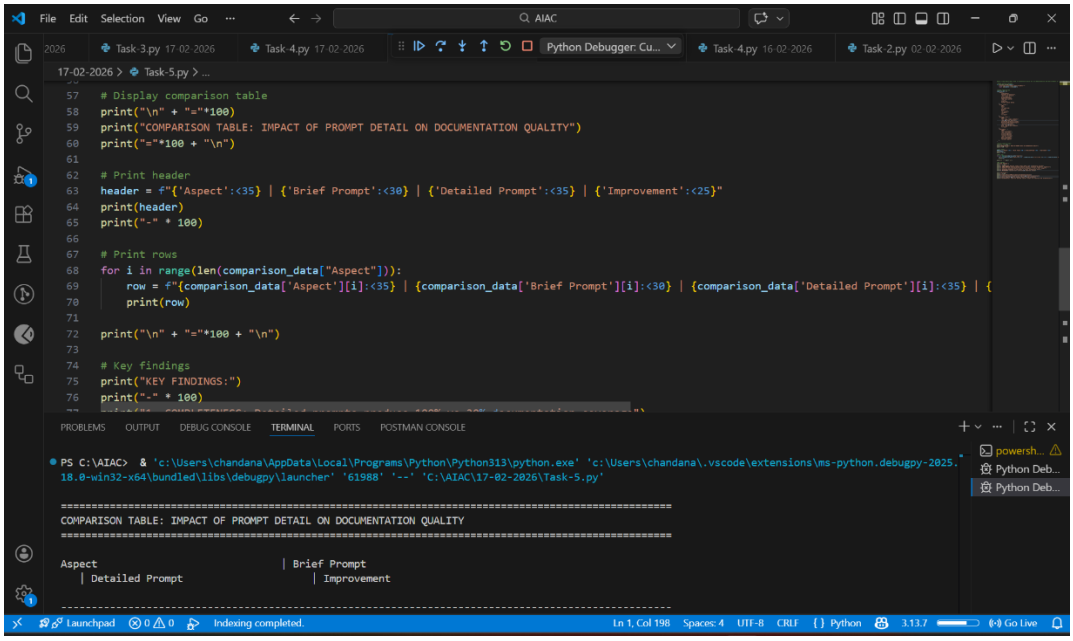
Create two prompts: one brief and one detailed.

• Use both prompts to document the same Python function.
• Compare the generated outputs.

Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

CODE:



```
# Display comparison table
print("\n" + "="*100)
print("COMPARISON TABLE: IMPACT OF PROMPT DETAIL ON DOCUMENTATION QUALITY")
print("="*100 + "\n")

# Print header
header = f"{'Aspect':<35} | {'Brief Prompt':<30} | {'Detailed Prompt':<35} | {'Improvement':<25}"
print(header)
print("-" * 100)

# Print rows
for i in range(len(comparison_data["Aspect"])):
    row = f"{comparison_data['Aspect'][i]:<35} | {comparison_data['Brief Prompt'][i]:<30} | {comparison_data['Detailed Prompt'][i]:<35} | {
    print(row)

print("\n" + "="*100 + "\n")

# Key findings
print("KEY FINDINGS:")
print("-" * 100)
```

```
PS C:\AIAC>  & 'c:\Users\chandana\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\chandana\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61988' '--' 'C:\AIAC\17-02-2026\Task-5.py'

====================================================================================================
COMPARISON TABLE: IMPACT OF PROMPT DETAIL ON DOCUMENTATION QUALITY
====================================================================================================

Aspect                              | Brief Prompt
   | Detailed Prompt                    | Improvement

----------------------------------------------------------------------------------------------------
```

ANALYSIS:

- The prompt asks AI to perform a comparative analysis task, examining how different levels of prompt detail affect documentation quality.
- AI understands that it must create two different prompts (brief vs. detailed) and generate documentation for the same Python function using both.
- The task focuses on evaluating differences in completeness, clarity, and accuracy, rather than modifying the function itself.
- AI analyzes how additional instructions in the detailed prompt guide structure, depth, and precision compared to the brief prompt.
- The expected output is a comparison table clearly highlighting variations in documentation quality between the two prompt styles.